Nestle Insight
Technical Implementation Guide

Team reference for architecture, repositories, setup, package selection, and relational schema

Project repositories: NestleInsight-backend, NestleInsight-docs, NestleInsight-AdminWeb, NestleInsight-mobile

Important note: the current GitHub screenshot shows the backend repository description as ASP.NET Core Web API. If the team is proceeding with NestJS, update the repository description later so the documentation, codebase, and demo all match.

1. Final technical stack

Mobile app: Flutter (Dart)

Admin / warehouse / planner web: React

Backend API: NestJS with Node.js and npm

Database: PostgreSQL

ORM / database toolkit: Prisma

Authentication: JWT + OTP email verification

API documentation: Swagger / OpenAPI from the backend

2. Repository structure and responsibility

Repository name	Main technology	Who uses it	Purpose
NestleInsight-mobile	Flutter + Dart	Sales Representatives, Territory Distributors, Shop Owners	Mobile application with role-based flows, offline caching, orders, deliveries, and day-end actions.
NestleInsight-AdminWeb	React + TypeScript/JavaScript	Admin, Warehouse Manager, Demand Planner	Web dashboard for user management, monitoring, warehouse operations, reports, and planning screens.
NestleInsight-backend	NestJS + Node.js + Prisma	All frontend clients indirectly	Single source of truth. Exposes APIs, applies business rules,

			handles authentication, and reads/writes PostgreSQL.
NestleInsight-docs	Markdown / DOCX / diagrams	Whole team	Contains UML, sprint plans, reports, architecture notes, testing artifacts, and presentation material.

Recommended rule: both the mobile app and web app must talk only to the backend API. They should never connect directly to PostgreSQL.

3. Languages and technologies by sector

Sector	Language(s)	Main tools	Why this choice fits
Mobile app	Dart	Flutter, dio, state management, go_router, local cache DB	One codebase for all mobile roles and easier shared UI components.
Web dashboard	JavaScript or TypeScript	React, Vite, router, UI library, data fetching	Fast dashboard development for admin, warehouse, and planner users.
Backend API	TypeScript	NestJS, Prisma, Swagger, Nodemailer	Clean modular architecture and consistent API development.
Database	SQL	PostgreSQL	Strong fit for relational data such as users, orders, deliveries, stock, and audit records.
Docs / diagrams	Markdown, PlantUML, DOCX	Draw.io, PlantUML, office tools	Supports reports, use cases, activity diagrams, and team communication.

4. How the system connects in development

Architecture flow: Flutter app / React web → NestJS backend API → PostgreSQL database

The database is usually on localhost only for the machine that runs it. During development, the backend can connect to PostgreSQL using localhost, while the web app calls the backend on localhost, and the Flutter app calls the backend using an emulator host or local network IP.

- Backend to database: localhost:5432 is fine if both run on the same laptop.
- React web to backend: usually http://localhost:3000 during development.

- Flutter Android emulator to backend: usually http://10.0.2.2:3000 instead of localhost.
- Flutter on a physical phone: use the laptop local IP address, for example http://192.168.x.x:3000.

5. What must be installed first (overall team setup)

Install first	Needed for	Required or optional	Notes
Git	All repositories	Required	Clone, commit, branch, and collaborate across the 4 repositories.
Node.js LTS + npm	NestJS backend and React web	Required	Install this before creating backend and web projects.
PostgreSQL	Database	Required	Use PostgreSQL for the agreed relational database layer.
VS Code	Coding	Recommended	Use extensions for Flutter, Dart, Prisma, ESLint, Prettier, and REST testing.
Flutter SDK + Android Studio / emulator tools	Mobile app	Required for mobile developers	Run flutter doctor and fix missing Android dependencies.
NestJS CLI	Backend scaffolding	Recommended	Speeds up module and resource creation.
Postman / Bruno / Thunder Client	API testing	Recommended	Test endpoints before frontend integration.
pgAdmin	Database inspection	Optional	Helpful GUI for viewing tables and rows.

6. Suggested installation order

1. Install Git.
2. Install Node.js LTS and verify node -v and npm -v.
3. Install PostgreSQL and create the project database.
4. Install VS Code and useful extensions.
5. Install Flutter SDK and confirm setup with flutter doctor.
6. Create the NestleInsight-backend repository project first and connect it to PostgreSQL.
7. Generate Prisma schema and run the first migration.
8. Create the NestleInsight-AdminWeb project and connect it to backend APIs.
9. Create the NestleInsight-mobile project and connect it to backend APIs.
10. Keep documentation, UML, and testing evidence in NestleInsight-docs.

7. Backend starter packages (NestleInsight-backend)

Install these first in the backend repository so the API, database connection, and documentation start in a clean way.

Package / tool	Why needed at the beginning	Typical use	Priority
@nestjs/common, @nestjs/core, @nestjs/platform-express	Base NestJS application	Controllers, modules, services	Immediate
prisma and @prisma/client	Database schema and typed queries	Connect NestJS to PostgreSQL	Immediate
@nestjs/config	Environment variable management	Read DATABASE_URL, JWT secret, mail config	Immediate
@nestjs/swagger and swagger-ui-express	API documentation	Generate docs for web/mobile/testing team	Immediate
@nestjs/jwt and passport-jwt	Authentication	Issue and verify JWT tokens	Immediate
class-validator and class-transformer	Request validation	Validate DTOs for login, orders, and shop creation	Immediate
nodemailer	Email sending	Send OTP to approved users	Early
bcrypt or bcryptjs	Password hashing if passwords are used	Secure stored credentials	Early
passport and @nestjs/passport	Auth strategy integration	JWT guards and role-based auth	Early
cors support via Nest app config	Web integration	Allow AdminWeb to call the backend locally	Early

8. Web starter packages (NestleInsight-AdminWeb)

Package / tool	Purpose	Why useful early	Priority
react + vite	Project foundation	Fast local development and simple setup	Immediate
react-router-dom	Navigation	Separate dashboards and role-based pages	Immediate
axios or fetch wrapper	API access	Call backend endpoints cleanly	Immediate
@tanstack/react-query	Server state management	Cache API responses and simplify loading states	Early
form library such as react-hook-form	Forms	User forms, allocation forms, and reports filters	Early

UI library such as MUI or Tailwind-based approach	Consistent interface	Speed up dashboard building	Early

9. Dart / Flutter packages to set up first (NestleInsight-mobile)

These are the first practical packages for the mobile app. The team does not need to add every possible package at once. Start with the core packages first.

Package	What it does	If not used	Why it helps	Setup order
dio	HTTP client for backend API calls	You would use the simpler http package or manual request handling	Better interceptors, token handling, timeout and error flow	1
flutter_bloc or provider	State management	UI logic can become mixed and hard to maintain	Keeps business logic cleaner and easier for team collaboration	2
go_router	Navigation and route control	Navigation becomes harder as app screens grow	Cleaner role-based routing and redirects	3
drift	Local relational storage for offline cache	Offline support becomes harder and local data handling gets messy	Good for cached shops, routes, and unsynced records	4
reactive_forms	Structured forms and validation	Large forms become repetitive and harder to validate	Useful for login, shop registration, and orders	5
fl_chart	Charts	You would need manual chart work or no charts	Simple future reporting visuals if needed later	Optional later

Recommended first Flutter setup: dio + one state management option (flutter_bloc or provider) + go_router. Add drift when offline caching starts, then add reactive_forms if forms become complex.

10. Example local development configuration

- Backend environment: PORT=3000 and DATABASE_URL=postgresql://user:password@localhost:5432/nestle_insight
- React environment: VITE_API_URL=http://localhost:3000
- Flutter emulator base URL: http://10.0.2.2:3000
- Flutter physical device base URL: use your laptop local IP on the same Wi-Fi network

11. Core backend modules to create early

- auth
- users
- roles / permissions
- territories
- shops
- products
- inventory
- orders
- deliveries
- payments
- feedback
- reports
- audit

Why this matters: the backend is the single source of truth for order flow, territory restrictions, stock allocation, delivery completion, returns, and user access rules.

12. Relational schema (high-level)

The database design should remain relational because the system has strong entity relationships between users, shops, territories, products, stock, orders, deliveries, and payments.

Table	Main purpose	Important fields	Key relationships
roles	Stores system role types	id, name	Linked to users
users	Stores login and account records	id, role_id, email, password_hash, is_approved	Belongs to roles; may link to territories and shops
territories	Regional operating areas	id, name	Linked to users, shops, warehouses, orders
warehouses	Regional warehouse records	id, territory_id, name	Belongs to territory
shops	Shop master data and assignments	id, territory_id, assigned_sales_rep_id, shop_code, status	Belongs to territory; may link to user account
shop_owner_accounts	Optional mapping for owner app access	id, user_id, shop_id	One owner account claims one assigned shop
products	Product master catalog	id, sku, name, unit, active	Linked to stock and order items
warehouse_stock	Current stock by warehouse and product	warehouse_id, product_id, quantity	Links warehouses and products
daily_allocations	Master daily stock	id, sales_rep_id,	Used for rep carrying

	allocations for sales reps	product_id, quantity, allocation_date	stock
stock_transactions	Stock movement ledger	id, warehouse_id, user_id, product_id, qty, type, reason	Tracks issue, return, damage, expiry, adjustments
orders	Order header	id, shop_id, created_by_user_id, status, payment_method, created_at	Belongs to shop; has many order items
order_items	Line items inside each order	id, order_id, product_id, quantity, delivered_quantity	Belongs to orders and products
order_status_history	Audit of status changes	id, order_id, from_status, to_status, changed_by, changed_at	Belongs to order
deliveries	Delivery assignment header	id, order_id, assigned_user_id, delivery_type, status	Links orders to sales reps or territory distributors
delivery_items	Delivered quantities by product	id, delivery_id, product_id, quantity	Belongs to deliveries
payments	Payment records	id, order_id, method, amount, collected_by_user_id, status	Belongs to order
feedback	Optional shop feedback	id, shop_id, user_id, message, created_at	Belongs to shop and user
audit_logs	System activity log	id, user_id, action, entity_type, entity_id, timestamp	Tracks sensitive changes across the system

13. Suggested relationship summary

- One role can have many users.
- One territory can have many shops, warehouses, and assigned users.
- One warehouse belongs to one territory but stores many products through warehouse_stock.
- One shop belongs to one territory and may be assigned to one sales representative.
- One shop may later be claimed by one shop owner account.
- One order belongs to one shop and contains many order_items.
- One order may have multiple status history records and one or more delivery records.
- One delivery is assigned either to a sales representative or a territory distributor.
- One stock transaction always records a product movement event for traceability.

14. Final implementation notes

- Keep the backend as the only component that talks directly to PostgreSQL.

- Lock the role-permission matrix early so web, mobile, and backend teams build the same rules.
- Create the order state flow and delivery flow before building many screens.
- Use seed data for demo accounts in all six roles.
- Keep the repository names exactly as: NestleInsight-backend, NestleInsight-docs, NestleInsight-AdminWeb, NestleInsight-mobile.